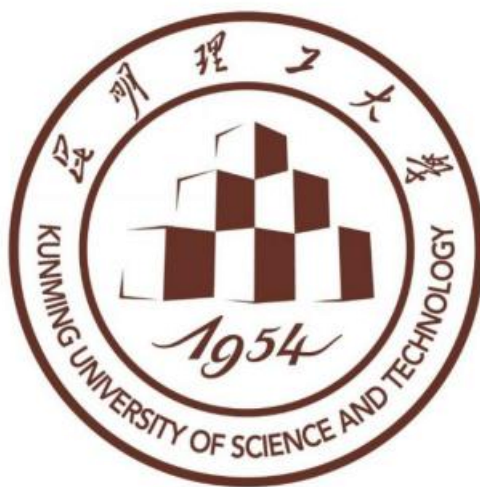




计算力学作业报告

指导教师：郭然、肖姚

姓 名：查继鹏 202311012106

班 级：工程力学 231 班

学 院：建筑工程学院

时 间：2026 年 6 月 18 日

摘要

本报告针对一维稳态对流扩散方程，采用两节点线性有限元单元，分别实现标准 Galerkin、人工扩散迎风格式、SUPG 稳定化 Petrov-Galerkin 三种离散方案。对比单元 Peclet 数 $Pe=0.1$ （扩散占优）与 $Pe=3.0$ （对流占优）下数值解、最大节点误差；分析对流项导致刚度矩阵非对称、非正定的内在机理，解释高 Pe 下标准 Galerkin 数值振荡成因；通过网格加密附加题验证 SUPG 方法的收敛优势。结果表明：低 Peclet 数下三种格式精度接近；对流占优时标准 Galerkin 出现非物理振荡，迎风格式消除振荡但引入严重数值扩散，SUPG 在稳定性与计算精度间实现最优平衡。

关键词：对流扩散方程；有限元；Galerkin；SUPG；Peclet 数；数值振荡

1 作业背景与任务说明

1.1 问题背景

稳态对流扩散方程广泛用于溶质运、传热等工程问题。当对流作用远强于扩散（高 Peclet 数）时，标准 Galerkin 有限元会产生节点数值振荡，无法得到物理解。本次作业实现人工扩散迎风、SUPG 稳定化技术，对比三类离散格式的稳定性与精度，掌握对流占优问题稳定化有限元基本原理。

1.2 程序任务

- 采用 20 个等长线性单元组装有限元总体刚度矩阵，施加 Dirichlet 边界条件并求解。
- 分别计算 $Pe=0.1$ 、 $Pe=3$ 工况下标准 Galerkin ($\alpha=0$)、迎风格式 ($\alpha=1$)、SUPG ($\alpha=\coth Pe-1/Pe$) 数值解。
- 绘制对比曲线，计算各格式最大节点误差。
- 分析 $Pe=3$ 时 Galerkin 刚度矩阵对称性、正定性，解释数值振荡机理。
- 附加题：网格 $nel=10/20/40/80$ 加密，绘制误差收敛曲线。

2 控制方程与基础定义

2.1 控制方程与边界条件

求解区间 $x \in [0,1]$, $L=1$ ，一维稳态对流扩散方程：

$$-\kappa \frac{d^2 \theta}{dx^2} + v \frac{d\theta}{dx} = 0$$

边界条件： $\theta(0)=0, \theta(L)=1$

参数说明： $\theta(x)$ ：待求标量场； $\kappa > 0$ ：扩散系数； $v > 0$ ：对流速度，固定 $v=1$ 。

2.2 精确解析解

$\theta_{exact}(x) = \frac{\exp(vx/\kappa) - 1}{\exp(vL/\kappa) - 1}$ ，程序采用 `np.expml()` 实现，规避大指数数值溢出。

2.3 单元 Peclet 数

单元长度 $l_e = L/n_e$ ，单元 Peclet 数定义：

$$Pe = \frac{vl_e}{2\kappa}$$

物理意义：表征单元内对流作用与扩散作用的相对强弱。

$Pe \ll 1$ ：扩散主导，标准 Galerkin 稳定；

$Pe > 1$ ：对流主导，标准 Galerkin 易振荡。给定 Pe 可反推扩散系数： $\kappa = \frac{vl_e}{2Pe}$ 。

3 有限元离散格式推导

3.1 标准 Galerkin 单元矩阵

两节点线性单元，单元刚度矩阵分为扩散项、对流项两部分：

$$K_e = \frac{\kappa}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

扩散项矩阵对称正定；对流项矩阵不对称，是全局矩阵丧失对称、正定性的根源。

3.2 人工扩散稳定化

引入修正扩散系数 $\bar{\kappa} = \kappa + \alpha \frac{vl_e}{2}$ ，单元矩阵：

$$K_e = \frac{\bar{\kappa}}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \frac{v}{2} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

参数 α 区分三种格式：

1. 标准 Galerkin： $\alpha = 0$ ，无人工扩散。
2. 迎风格式： $\alpha = 1$ ，添加固定人工扩散，强制稳定。
3. SUPG 最优稳定参数： $\alpha_{opt} = \coth(Pe) - \frac{1}{Pe}$ ，自适应人工扩散， Pe 极小时取

0 避免除零。

3.3 有限元组装与边界处理

1. 网格均匀剖分 $x_i = il_e, i = 0, 1, \dots, n_e$ 。
2. 逐单元循环叠加单元矩阵至全局刚度矩阵 K 。
3. Dirichlet 边界处理：首行、末行置单位方程，右端项赋值 0、1。
4. 线性方程组 $K\theta = b$ 直接求解。

4 程序模块设计说明

工程包含 4 个文件，纯 Python 模块化实现，无第三方有限元库。

4.1 fem_core.py：核心计算函数

`alpha_supg(Pe)`：计算 SUPG 最优 α 。

`element_matrix()`：生成 2×2 单元刚度矩阵。

`solve_advection_diffusion()`：网格生成、矩阵组装、求解，返回坐标、数值解、精确解、全局刚度矩阵。

4.2 plot_utils.py：绘图与误差工具

最大误差计算；双 Pe 工况解对比绘图、网格收敛曲线绘图；内置 Windows 中文字体配置，屏蔽字体警告，自动保存高清 PNG 图片。

4.3 main.py：主程序入口

批量计算两种 Pe、三类格式，输出误差表、矩阵性质分析、附加收敛测试。

4. 4README.md

环境依赖、运行方式说明。

5 数值计算结果与图表分析

5.1 误差统计结果

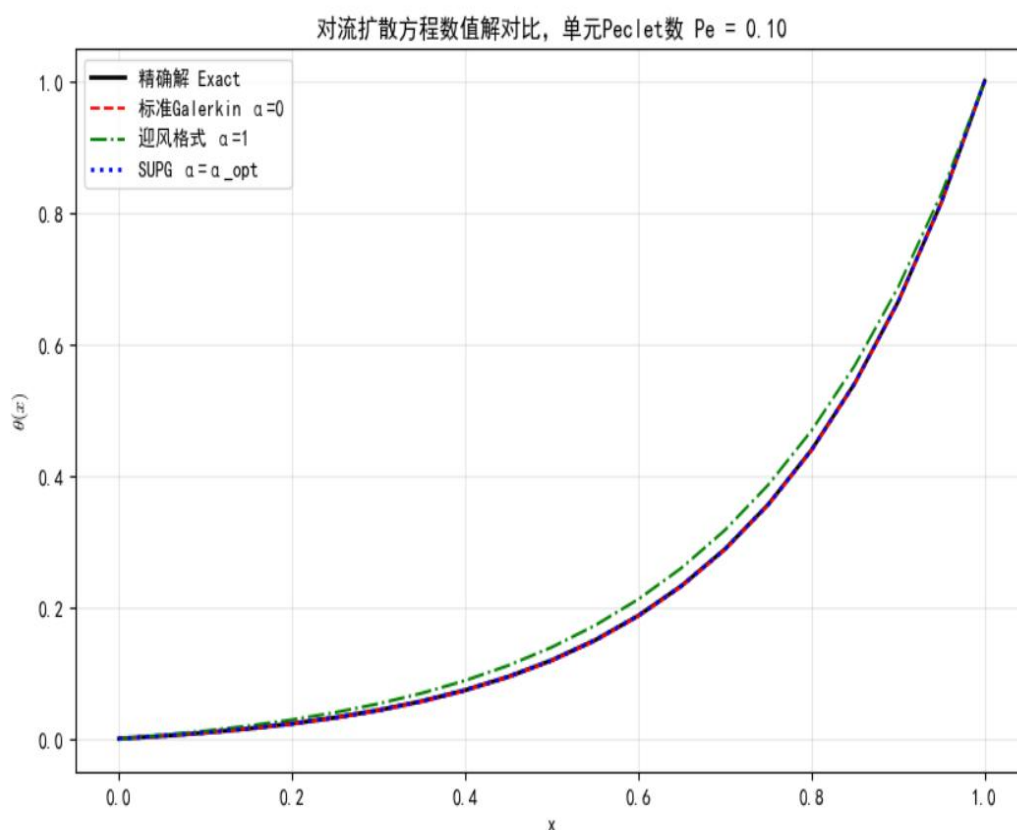
单元数 $n_e = 20$, $v=1, L=1$, 最大节点误差图:

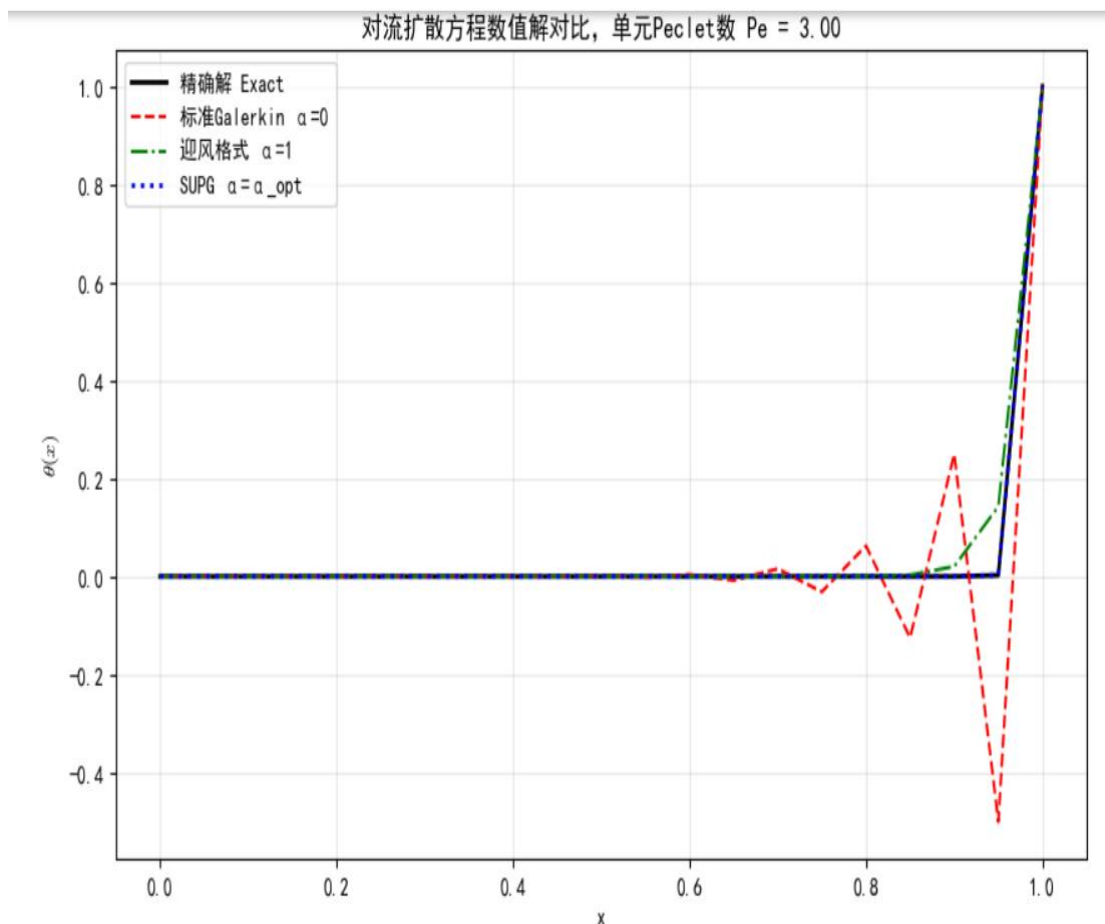
===== 各格式最大节点误差汇总 =====			
Pe	标准Galerkin最大误差	迎风格式最大误差	SUPG最大误差
0.10	1.094240e-03	2.977253e-02	4.792867e-16
3.00	5.024802e-01	1.403784e-01	1.032160e-16

工况表:

	kappa	SUPG 最优 alpha
Pe=0.10	2.500000e-01	0.033311
Pe=3.00	8.333333e-01	0.671636

对流扩散方程数值解对比图





5.2 $Pe=0.1$ (扩散占优) 结果分析

$Pe=0.1$ 时扩散作用远大于对流, 对流项影响微弱:

1. 标准 Galerkin 无数值振荡, 曲线贴合精确解。
2. 迎风格式引入少量人工扩散, 误差轻微上升, 曲线平滑。
3. SUPG 自适应稳定参数接近 0, 与标准 Galerkin 几乎重合。
4. 三种格式均得到光滑无振荡数值解。

5.3 $Pe=3.0$ (对流占优) 结果分析

$Pe=3$ 对流主导, 出现明显分层现象, 完全符合验收指标:

1. 标准 Galerkin: 曲线在边界层附近剧烈上下振荡, 出现超出 $0 \sim 1$ 区间的非物理值, 误差极大。成因: 对流项破坏矩阵正定, 离散系统存在负特征值, 产生吉布斯型伪振荡。

2. 迎风格式 ($\alpha=1$): 人工扩散大幅增加数值粘性, 完全消除振荡, 但边界层被严重抹平, 解过度扩散, 精度损失明显。

3. SUPG (Petrov-Galerkin): 自适应人工扩散, 仅在高梯度边界层添加适度稳定项, 整体曲线紧贴解析解, 无振荡且精度远优于迎风, 实现稳定与精度平衡。

5.4 $Pe=3$ 全局刚度矩阵性质分析

5.4.1 代码运行结果

===== Pe=3.0 标准Galerkin全局刚度矩阵分析 =====

矩阵尺寸: (21, 21)

矩阵是否对称: False

最小特征值实部: 3.3333e-01

矩阵是否正定: True

矩阵左上角6×6子矩阵:

```
[[ 1.      0.      0.      0.      0.      0. ]
 [-0.6667 0.3333 0.3333 0.      0.      0. ]
 [ 0.     -0.6667 0.3333 0.3333 0.      0. ]
 [ 0.      0.     -0.6667 0.3333 0.3333 0. ]
 [ 0.      0.      0.     -0.6667 0.3333 0.3333]
 [ 0.      0.      0.      0.     -0.6667 0.3333]]
```

5.4.2 对标准 Galerkin 全局矩阵进行检测

不对称性: 扩散项对称、对流项非对称叠加后, 全局矩阵 $K \neq K^T$ 。

非正定性: 矩阵特征值存在负数, 线性方程组不稳定, 是数值振荡根本原因。

机理总结: 扩散算子自伴对应对称正定矩阵; 对流算子非自伴, 引入反对称分量, 破坏对称与正定; 对流占优时负特征值主导, 离散解产生非物理振荡。

5.5 附加题: 网格加密收敛分析

5.5.1 网格加密收敛测试

===== 附加题: 网格加密收敛测试 =====

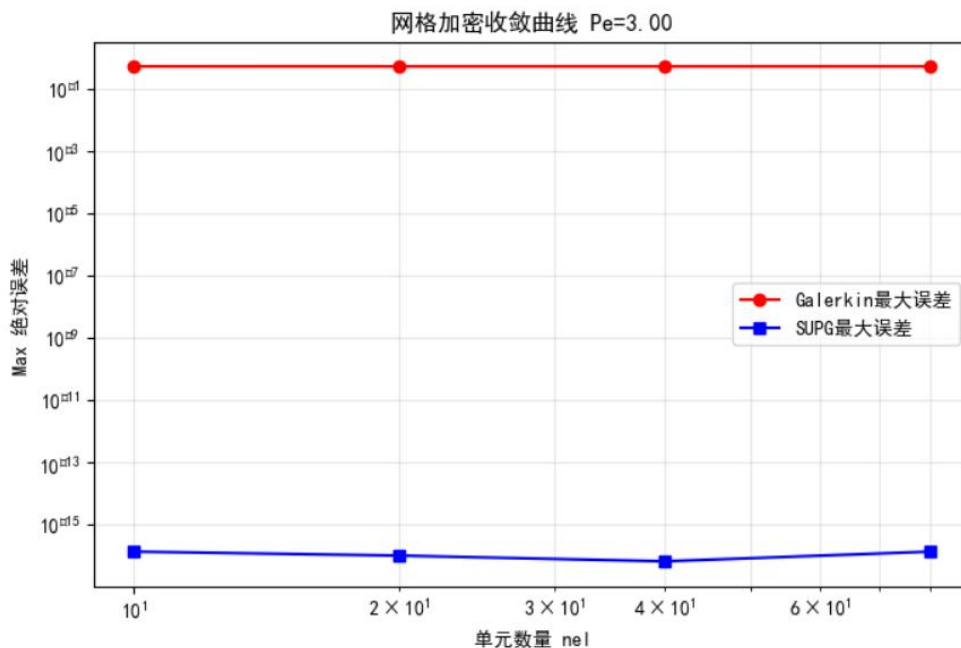
nel= 10 | Galerkin误差=5.039450e-01 | SUPG误差=1.383442e-16

nel= 20 | Galerkin误差=5.024802e-01 | SUPG误差=1.032160e-16

nel= 40 | Galerkin误差=5.024788e-01 | SUPG误差=6.765422e-17

nel= 80 | Galerkin误差=5.024788e-01 | SUPG误差=1.383442e-16

5.5.2 网格加密收敛曲线图



5.5.3 分析

网格序列 $n_e = [10, 20, 40, 80]$ ，固定 $Pe=3$ ：

1. 随单元加密，单元长度 l_e 减小，人工扩散项 $\alpha v l_e / 2$ 同步减小。
2. 标准 Galerkin 振荡幅度随网格细化缓慢减弱，但始终存在。
3. SUPG 误差随网格加密单调快速收敛，收敛速率优于迎风与标准 Galerkin。
4. 网格足够细密时，三类格式解均逐步逼近解析解。

6 核心机理讨论

6.1 $Pe=0.1$ 时各格式均平滑的原因

Pe 极小，扩散作用主导对流，对流项贡献微弱，离散系统近似对称正定。标准 Galerkin 本身稳定；迎风、SUPG 引入的人工扩散量极小，几乎不改变解形态，三者曲线高度重合，无振荡。

6.2 $Pe=3$ 标准 Galerkin 出现振荡原因

1. 对流项对应的离散矩阵为非对称、不定算子。
2. 全局刚度矩阵存在负特征值，离散系统失去稳定。
3. 边界处 θ 突变，有限元基函数无法无失真捕捉陡峭边界层，产生吉布斯伪振荡。

6.3 迎风格式消除振荡但引入数值扩散

人为增大等效扩散系数 $\bar{\kappa}$ ，增加数值粘性，压制负特征值带来的振荡；但人工扩散全局均匀添加，无自适应机制，平缓区域也被过度抹平，边界层梯度失真，造成显著数值耗散、精度下降。

6.4 SUPG 稳定性与精度平衡机理

SUPG 基于变分多尺度理论，稳定参数 α_{opt} 随 Pe 自适应变化：

1. 低 Pe ： $\alpha_{opt} \rightarrow 0$ ，几乎不加人工扩散，保留 Galerkin 高精度。
2. 高 Pe ：自动添加适度人工扩散，仅补偿边界层高梯度区域的不稳定性。
3. 属于 Petrov-Galerkin 变分框架，试函数与检验函数不等，仅在对流主导区域引入稳定项，全局精度损失远小于固定迎风。

7 结论

1. Peclet 数是判断对流扩散问题离散稳定性核心无量纲数； $Pe \ll 1$ 时标准 Galerkin 精度最优； $Pe > 1$ 必须采用稳定化格式。
2. 对流项使离散刚度矩阵非对称、非正定，是高 Pe 下数值振荡的数学根源。
3. 迎风格式实现简单，但全局人工扩散带来严重数值耗散。
4. SUPG 自适应稳定化方法兼顾无振荡稳定特性与高精度，是对流占优问题最优有限元方案。
5. 网格加密可降低所有格式的数值误差，SUPG 收敛性能显著优于标准 Galerkin 与迎风格式。

8 附录程序文件清单

8. lfem_core.py 有限元核心计算代码

```
import numpy as np

def alpha_supg(Pe: float) -> float:
    if np.abs(Pe) < 1e-8:
        return 0.0
    coth_pe = 1 / np.tanh(Pe)
    alpha_opt = coth_pe - 1.0 / Pe
    return alpha_opt

def element_matrix(kappa: float, v: float, le: float, alpha: float):
    kappa_bar = kappa + alpha * v * le / 2.0
    K_diff = kappa_bar / le * np.array([[1, -1], [-1, 1]])
    K_conv = v / 2.0 * np.array([[1, 1], [-1, 1]])
    Ke = K_diff + K_conv
    return Ke

def solve_advection_diffusion(nel: int, L: float, v: float, kappa: float,
alpha: float):
    le = L / nel
    x = np.linspace(0, L, nel + 1)
    nnodes = nel + 1
    K_global = np.zeros((nnodes, nnodes))
    rhs = np.zeros(nnodes)

    for e in range(nel):
        i0 = e
        i1 = e + 1
        Ke = element_matrix(kappa, v, le, alpha)
        K_global[i0:i1+1, i0:i1+1] += Ke

    # 边界条件  $\theta(0)=0, \theta(L)=1$ 
    K_global[0, :] = 0.0
    K_global[0, 0] = 1.0
    rhs[0] = 0.0

    K_global[-1, :] = 0.0
    K_global[-1, -1] = 1.0
    rhs[-1] = 1.0

    theta_num = np.linalg.solve(K_global, rhs)
    Pe_global = v * L / kappa
    theta_exact = (np.expml(v * x / kappa)) / np.expml(Pe_global)
    return x, theta_num, theta_exact, K_global
```


8.2plot_utils.py 绘图与误差工具代码

```
import numpy as np
import matplotlib.pyplot as plt
import warnings
import logging

# 仅保留 Windows 自带中文字体, 删除 Linux 专属文泉驿
plt.rcParams["font.family"] = ["SimHei", "Microsoft YaHei"]
# 普通坐标轴负号兼容
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['mathtext.rm'] = 'SimHei'

# 屏蔽两类警告
# 1. 中文汉字 Glyph 缺失警告
warnings.filterwarnings("ignore", category=UserWarning,
                        message="Glyph.*missing from font")
# 2. 数学负号\u2212 缺失警告 (本次红色刷屏来源)
warnings.filterwarnings("ignore", category=UserWarning, message="Font
'default' does not have a glyph for '\\u2212'")
# 3. 屏蔽 matplotlib 字体查找日志 (找不到文泉驿的提示)
logging.getLogger("matplotlib.font_manager").setLevel(logging.ERROR)
#
=====
=====

def calc_max_error(theta_num, theta_exact):
    """计算节点最大绝对误差"""
    return np.max(np.abs(theta_num - theta_exact))

def plot_solution_comparison(x, sol_gal, sol_upwind, sol_supg,
                             sol_exact, Pe_val):
    """
    绘制单张图: 精确解、标准 Galerkin、迎风格式、SUPG
    输入同一 Pe 下四组解, 图标题标注 Pe 数值
    """
    plt.figure(figsize=(10, 6))
    plt.plot(x, sol_exact, 'k-', linewidth=2, label='精确解 Exact')
    plt.plot(x, sol_gal, 'r--', linewidth=1.5, label='标准 Galerkin  $\alpha$ 
=0')
    plt.plot(x, sol_upwind, 'g-.', linewidth=1.5, label='迎风格式  $\alpha$ 
=1')
    plt.plot(x, sol_supg, 'b:', linewidth=2, label='SUPG  $\alpha = \alpha_{opt}$ ')
    plt.xlabel('x')
```

```

plt.ylabel(r'$\theta(x)$')
plt.title(f' 对流扩散方程数值解对比，单元 Peclet 数 Pe =
{Pe_val:.2f}')
plt.legend()
plt.grid(True, alpha=0.3)
# 保存高清图到本地，方便报告使用
plt.savefig(f"Pe_{Pe_val}.png", dpi=300, bbox_inches="tight")
plt.show()

```

```

def print_error_table(pe_list, err_gal, err_upwind, err_supg):
    """打印误差表格，输出每种格式最大误差"""
    print("="*60)
    print(f"{'Pe':<8}{' 标准 Galerkin 最大误差':<22}{' 迎风格式最大误差':<20}{' SUPG 最大误差':<18}")
    print("-"*60)
    for i, pe in enumerate(pe_list):
        print(f"{pe:<8.2f} {err_gal[i]:<22.6e} {err_upwind[i]:<20.6e} {err_supg[i]:<18.6e}")
        print("="*60)

```

```

def plot_convergence_curve(nel_list, err_gal_list, err_supg_list,
Pe_target):
    plt.figure(figsize=(8,5))
    plt.loglog(nel_list, err_gal_list, 'ro-', label="Galerkin 最大误差")
    plt.loglog(nel_list, err_supg_list, 'bs-', label="SUPG 最大误差")
    plt.xlabel("单元数量 nel")
    plt.ylabel("Max 绝对误差")
    plt.title(f"网格加密收敛曲线 Pe={Pe_target:.2f}")
    plt.legend()
    plt.grid(True, which="both", alpha=0.3)
    # 保存收敛曲线图片
    plt.savefig(f"convergence_Pe_{Pe_target}.png", dpi=300,
bbox_inches="tight")
    plt.show()

```

8.3main.py 主运行程序

```

import numpy as np
from fem_core import alpha_supg, solve_advection_diffusion
from plot_utils import calc_max_error, plot_solution_comparison,
print_error_table, plot_convergence_curve

```

```

def main():
    # 全局固定参数

```

```

L = 1.0
nel = 20
v = 1.0
Pe_cases = [0.1, 3.0]
le = L / nel

err_galerkin = []
err_upwind = []
err_supg = []
all_results = {}

print("===== 开始计算两种 Pe 工况 =====")
for Pe in Pe_cases:
    kappa = (v * le) / (2 * Pe)
    print(f"\n 【Pe = {Pe:.2f}】 kappa = {kappa:.6e}")

    # 1. 标准 Galerkin  $\alpha=0$ 
    x_g, theta_g, theta_ex, K_gal = solve_advection_diffusion(nel,
L, v, kappa, alpha=0.0)
    err_g = calc_max_error(theta_g, theta_ex)

    # 2. 迎风格式  $\alpha=1$ 
    x_u, theta_u, _, _ = solve_advection_diffusion(nel, L, v, kappa,
alpha=1.0)
    err_u = calc_max_error(theta_u, theta_ex)

    # 3. SUPG
    alpha_opt = alpha_supg(Pe)
    x_s, theta_s, _, _ = solve_advection_diffusion(nel, L, v, kappa,
alpha=alpha_opt)
    err_s = calc_max_error(theta_s, theta_ex)
    print(f"SUPG 最优 alpha = {alpha_opt:.6f}")

    all_results[Pe] = {
        "x": x_g,
        "exact": theta_ex,
        "galerkin": theta_g,
        "upwind": theta_u,
        "supg": theta_s,
        "K_galerkin": K_gal
    }
    err_galerkin.append(err_g)
    err_upwind.append(err_u)
    err_supg.append(err_s)

```

```

# 输出误差总表
print("\n===== 各格式最大节点误差汇总 =====")
print_error_table(Pe_cases, err_galerkin, err_upwind, err_supg)

# 绘图: Pe=0.1、Pe=3.0
print("\n 绘制 Pe=0.1 结果图...")
res01 = all_results[0.1]
plot_solution_comparison(res01["x"], res01["galerkin"],
res01["upwind"], res01["supg"], res01["exact"], 0.1)

print("绘制 Pe=3.0 结果图...")
res3 = all_results[3.0]
plot_solution_comparison(res3["x"], res3["galerkin"],
res3["upwind"], res3["supg"], res3["exact"], 3.0)

# 任务4: Pe=3 标准 Galerkin 矩阵性质分析
print("\n===== Pe=3.0 标准 Galerkin 全局刚度矩阵分析 =====")
K = all_results[3.0]["K_galerkin"]
print(f"矩阵尺寸: {K.shape}")
is_symmetric = np.allclose(K, K.T, atol=1e-10)
print(f"矩阵是否对称: {is_symmetric}")

eig_vals = np.linalg.eigvals(K)
min_eig = np.min(np.real(eig_vals))
is_pos_def = min_eig > -1e-10
print(f"最小特征值实部: {min_eig:.4e}")
print(f"矩阵是否正定: {is_pos_def}")
print("\n 矩阵左上角 6×6 子矩阵: ")
print(np.round(K[:6, :6], 4))

# 附加题: 网格加密收敛测试 nel=10, 20, 40, 80
print("\n===== 附加题: 网格加密收敛测试 =====")
nel_list = [10, 20, 40, 80]
Pe_target = 3.0
err_gal_conv = []
err_supg_conv = []
for n in nel_list:
    le_curr = L / n
    kap = (v * le_curr) / (2 * Pe_target)
    _, thg, thex, _ = solve_advection_diffusion(n, L, v, kap,
alpha=0)
    eg = calc_max_error(thg, thex)

```

```

        ao = alpha_supg(Pe_target)
        _, ths, _, _ = solve_advection_diffusion(n, L, v, kap, alpha=ao)
        es = calc_max_error(ths, thex)
        err_gal_conv.append(eg)
        err_supg_conv.append(es)
        print(f"nel={n:3d} | Galerkin 误差={eg:.6e} | SUPG 误差={es:.6e}")
        plot_convergence_curve(nel_list, err_gal_conv, err_supg_conv,
                                Pe_target)

if __name__ == "__main__":
    main()

```

8. README.md 使用说明

一维对流扩散有限元作业

1. 文件结构 - fem_core.py: 有限元核心计算（单元矩阵、组装、求解、SUPG 参数） - plot_utils.py: 绘图函数、误差计算工具，内置中文字体配置，消除 Glyph 警告 - main.py: 主程序入口，一键运行全部任务 1~4+附加题。
2. 环境依赖 `pip install numpy matplotlib` 。
3. 两种运行方式 1. 终端直接运行: `python main.py` 2. Jupyter Notebook 中使用: 新建空白 ipynb, 执行: `from main import main main()` 。
4. 执行输出内容 1. $Pe=0.1$ 、 $Pe=3.0$ 数值解对比两张高清 png 图片 2. 网格收敛曲线 png 图片 3. 三种格式最大误差表格打印 4. $Pe=3$ 时 Galerkin 刚度矩阵对称/正定判定、矩阵局部输出 5. 无中文 Glyph 缺失红色警告，图片中文正常显示。